

Relazione Water Potability

Patrick Pasolini Vitali
Machine Learning and Data Mining

Indice

1	Introduzione	2
1.1	Descrizione del problema	2
1.2	Dataset e features	2
1.3	Obiettivo	3
2	Esplorazione e analisi dei dati	4
2.1	Controlli di integrità e pulizia pre-split	4
2.2	Suddivisione in training e test set	4
2.3	Distribuzione delle feature	5
2.4	Analisi delle correlazioni	8
2.5	Valori mancanti	9
3	Preprocessing e preparazione dei dati	11
3.1	Pipeline	11
3.2	Imputazione dei missing value	11
3.3	Scaling delle feature	11
3.4	Gestione dello sbilanciamento delle classi	11
4	Modelli	13
4.1	Scelta dei modelli da ottimizzare	13
4.2	Scelta iperparametri con Grid Search	13
4.3	Random Forest	14
4.4	XGBoost	16
4.5	SVM RBF	18
4.6	MLP	20
4.7	Rete neurale con Optuna	22
4.8	Confronto modelli con e senza bilanciamento	24
4.9	Modello migliore	25
5	Conclusioni	26

1 Introduzione

Il seguente progetto consiste nello sviluppo di modelli di machine learning relativamente al problema descritto nel paragrafo successivo, grazie alle conoscenze apprese durante il corso di Machine Learning and Data Mining.

1.1 Descrizione del problema

Il progetto affronta un problema di **classificazione binaria** nel dominio della qualità dell'acqua: data una serie di misure chimico-fisiche di un campione, l'obiettivo è prevedere se l'acqua sia **potabile** o **non potabile**. Si tratta di un task di rilevanza pratica, poiché valutare automaticamente la potabilità a partire da parametri misurabili supporta il monitoraggio ambientale e la tutela della salute pubblica.

1.2 Dataset e features

Il dataset è composto da 3276 istanze, 9 feature di tipo float e una feature classe di tipo binario:

1. **pH**: parametro che valuta il bilancio acido-base dell'acqua, indicandone lo stato di acidità o alcalinità. L'Organizzazione Mondiale della Sanità (OMS) raccomanda un intervallo tra 6,5 e 8,5. I valori dell'indagine rientrano in tale intervallo degli standard OMS.
2. **Hardness**: la durezza è causata principalmente dai sali di calcio e magnesio, che si dissolvono dai depositi geologici attraverso cui scorre l'acqua. Maggiore è il tempo di contatto dell'acqua con tali materiali, maggiore è la durezza presente. Storicamente la durezza era definita come la capacità dell'acqua di far precipitare il sapone a causa di calcio e magnesio.
3. **Solids (TDS)**: l'acqua è in grado di dissolvere un'ampia gamma di minerali e sali inorganici (potassio, calcio, sodio, bicarbonati, cloruri, magnesio). Un valore elevato di TDS (*Total Dissolved Solids*) indica un'acqua altamente mineralizzata e ne peggiora il sapore. Il limite desiderabile è 500 mg/L, quello massimo 1000 mg/L per uso potabile.
4. **Chloramines**: le clorammine sono disinfettanti a base di cloro (ottenuti combinando cloro e ammoniaca) impiegati nei sistemi idrici pubblici per eliminare batteri e microrganismi e mantenere l'acqua sicura lungo la rete di distribuzione. Sono preferite al solo cloro perché più stabili e persistenti. Concentrazioni fino a 4 mg/L (circa 4 ppm) sono considerate sicure nell'acqua potabile.
5. **Sulfate**: i solfati sono sostanze naturali presenti in minerali, suolo e rocce. La loro concentrazione varia da 3 a 30 mg/L nella maggior parte delle riserve di acqua dolce, fino a oltre 1000 mg/L in alcune zone, e raggiunge circa 2700 mg/L nell'acqua di mare. Per l'acqua potabile l'UE fissa un valore di parametro di **250 mg/L**: oltre tale soglia compaiono sapore sgradevole ed effetti lassativi.
6. **Conductivity**: la conducibilità elettrica misura la capacità dell'acqua di trasmettere corrente, capacità che dipende dalla concentrazione di ioni disciolti. L'acqua pura è un buon isolante, ma all'aumentare dei sali disciolti aumenta anche la conducibilità; per questo essa è un indicatore indiretto della quantità di solidi disciolti. Secondo gli standard OMS il valore non dovrebbe superare i 400 $\mu\text{S}/\text{cm}$.
7. **Organic_carbon (TOC)**: il carbonio organico totale deriva dalla decomposizione di materia organica naturale e da fonti sintetiche, e misura la quantità complessiva di carbonio nei composti organici presenti nell'acqua. L'UE classifica il TOC come parametro indicatore, per cui non fissa una soglia numerica fissa ma richiede l'assenza di variazioni anomale rispetto ai valori abituali.

8. **Trihalomethanes (THM)**: i trihalometani sono sottoprodotti che si formano nell'acqua trattata con cloro. La loro concentrazione varia in base al contenuto di materia organica, alla quantità di cloro impiegata e alla temperatura dell'acqua durante il trattamento. La Direttiva UE fissa un valore massimo di $100 \mu\text{g/L}$ per la somma dei THM (in Italia ridotto a $30 \mu\text{g/L}$).
9. **Turbidity**: misura quanto l'acqua appare "torbida" a causa delle particelle solide in sospensione: più materiale sospeso è presente, più l'acqua diffonde la luce invece di lasciarla passare. È quindi un indicatore visivo della qualità dell'acqua e dell'efficacia del trattamento. L'UE fissa un valore di **1,0 NTU** per l'acqua in uscita dall'impianto di trattamento.
10. **Potability**: indica se l'acqua è sicura per il consumo umano, dove 1 significa potabile e 0 non potabile.

1.3 Obiettivo

L'obiettivo dei modelli implementati è quello di prevedere con precisione la potabilità dell'acqua sulla base delle proprietà chimico-fisiche dei campioni.

2 Esplorazione e analisi dei dati

L'analisi esplorativa ha l'obiettivo di comprendere la struttura del dataset, valutare la qualità dei dati e individuare eventuali criticità che potessero condizionare le fasi successive di modellazione.

Un principio metodologico ha guidato l'intero capitolo: la separazione tra training set e test set è stata effettuata il prima possibile e ogni analisi che comporti la stima di parametri dai dati è stata condotta esclusivamente sul training set. In questo modo si evita qualsiasi forma di *data leakage*, ossia il trasferimento involontario di informazioni dal test set al processo di apprendimento, che porterebbe a una stima ottimistica e non realistica delle prestazioni.

2.1 Controlli di integrità e pulizia pre-split

Prima della suddivisione del dataset sono stati eseguiti unicamente controlli di natura deterministica, che non stimano alcun parametro a partire dalle osservazioni e risultano pertanto leciti sull'intero dataset. Sono stati verificati i tipi di dato delle colonne, l'eventuale presenza di feature costanti (a varianza nulla) o duplicate, e la presenza di righe identiche. In questa fase si è proceduto alla rimozione dei duplicati esatti. La scelta di effettuarla prima dello split è motivata dal fatto che la presenza della stessa istanza sia nel training sia nel test costituirebbe una forma di leakage: il modello verrebbe valutato su osservazioni già viste nella fase di training. Nel dataset in esame non sono comunque emersi duplicati, né colonne costanti o ridondanti.

2.2 Suddivisione in training e test set

Il dataset è stato suddiviso in training set (75%) e in test set (25%), adottando una suddivisione stratificata rispetto alla variabile target. La stratificazione garantisce che la proporzione tra le due classi (61% non potabile, 39% potabile) sia preservata in entrambi i sottoinsiemi, condizione importante in presenza di un lieve sbilanciamento. Il test set è stato accantonato ed è stato utilizzato per la valutazione finale dei modelli ottimizzati.

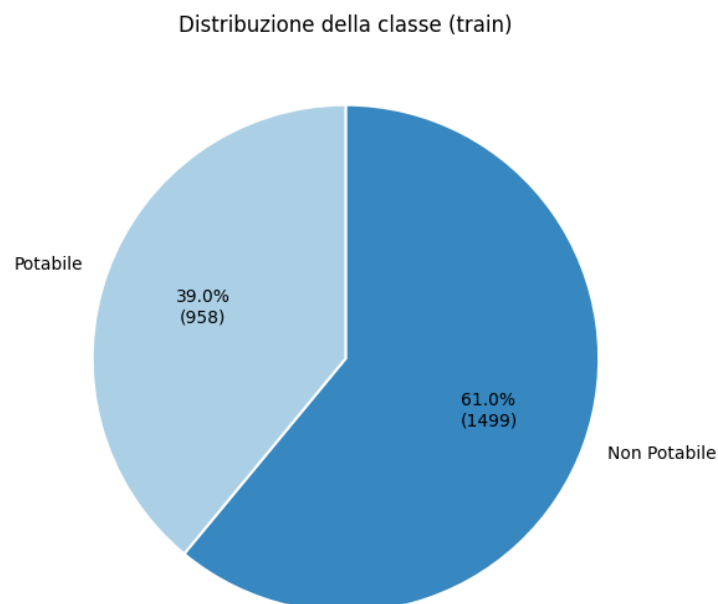


Figura 1: Suddivisione stratificata in training e test set.

2.3 Distribuzione delle feature

La distribuzione delle nove variabili è stata esaminata da tre punti di vista complementari:

1. **Dispersione per classe:** riportando ciascuna feature in funzione dell'indice di osservazione e distinguendo i punti in base alla potabilità (Figura 2), le due classi risultano completamente sovrapposte su tutte le variabili. Non si individua alcun intervallo di valori, né alcuna regione del grafico, in cui una delle due classi prevalga sull'altra: i punti potabili e non potabili occupano in modo indistinguibile lo stesso supporto. Già a livello visivo, dunque, nessuna feature presa singolarmente sembra in grado di separare le classi.

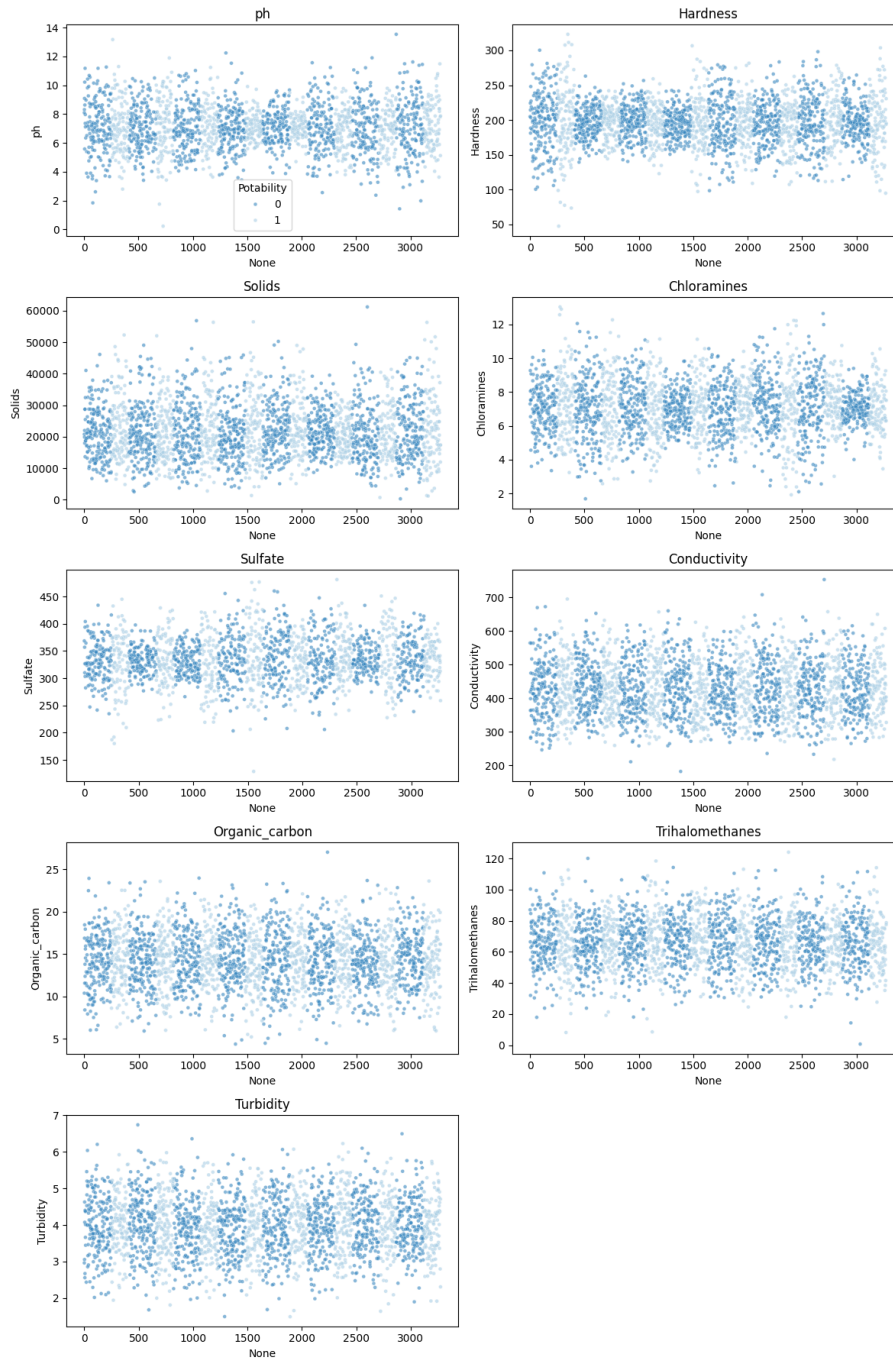


Figura 2: Dispersione delle feature per classe.

2. **Distribuzioni condizionate alla classe (istogrammi).** Gli istogrammi sovrapposti per classe, con relativa stima di densità (Figura 3), confermano questa osservazione. Per ogni feature, le distribuzioni della classe potabile e di quella non potabile presentano forma, posizione e dispersione pressoché coincidenti: le curve di densità si sovrappongono quasi perfettamente, e l'unica differenza apprezzabile è l'altezza complessiva, riconducibile al lieve sbilanciamento numerico tra le due classi. Le distribuzioni risultano inoltre tutte approssimativamente simmetriche, prive di code anomale. Questa regolarità, unita alla totale assenza di separazione tra le classi, è coerente con il risultato emerso dall'analisi delle correlazioni e anticipa la difficoltà intrinseca del problema di classificazione.

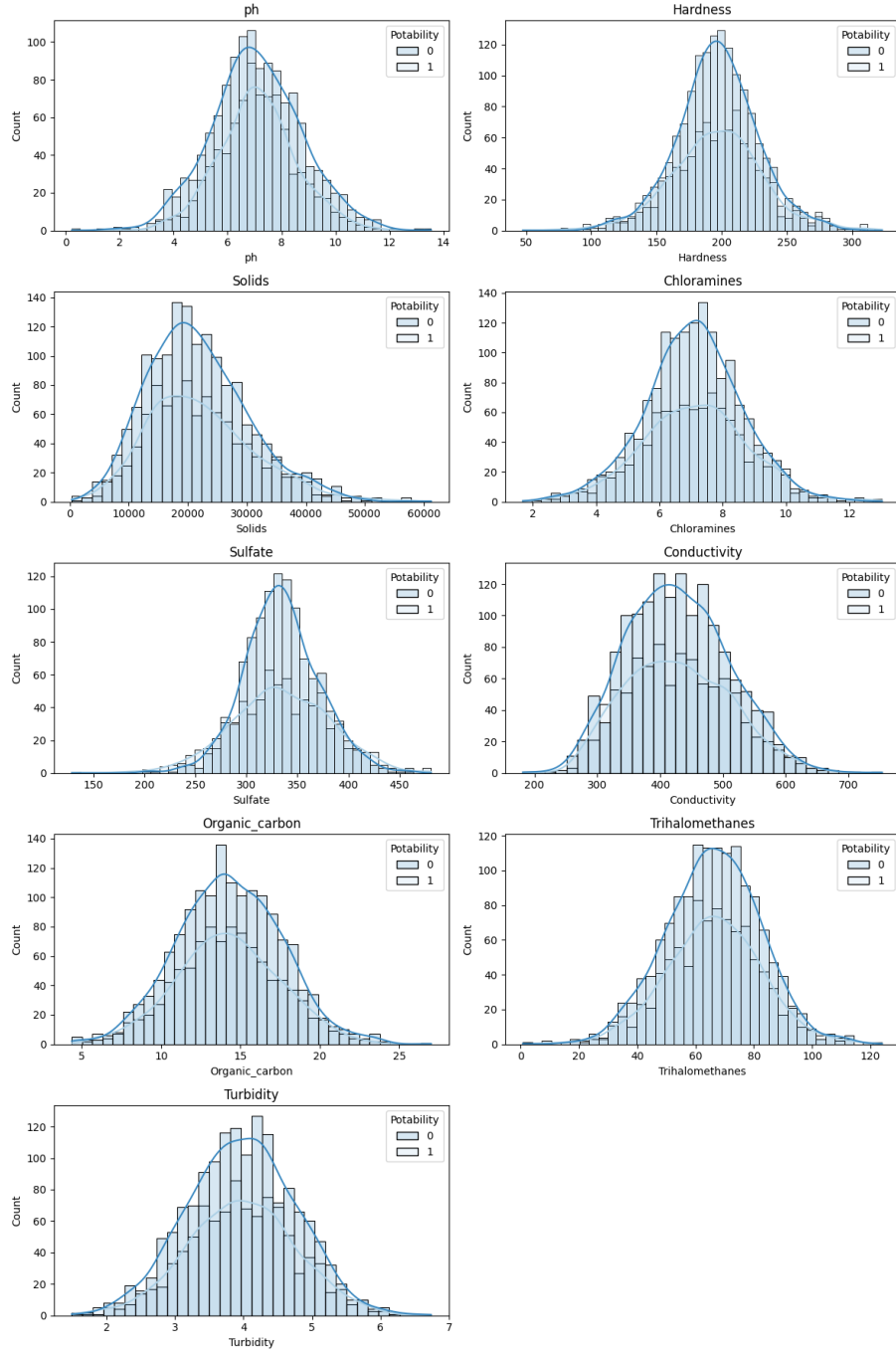


Figura 3: Iistogrammi delle distribuzioni condizionate alla classe.

3. I boxplot in Figura 4 mostrano, per quasi tutte le feature, la presenza di *outlier* collocati su entrambe le code della distribuzione; il fenomeno è particolarmente marcato per la feature “Solid”, che presenta numerose osservazioni nella coda superiore. L’analisi non ha tuttavia evidenziato valori fisicamente impossibili: il pH si mantiene nell’intervallo ammissibile $[0, 14]$ e nessuna concentrazione assume valori negativi. Si è quindi scelto di **non** rimuovere tali osservazioni: la loro eliminazione avrebbe comportato la perdita di un numero consistente di istanze e introdotto uno sbilanciamento artificiale tra le classi, a fronte di un beneficio non giustificato trattandosi di valori ammissibili. La presenza di code marcate ha invece motivato, nelle fasi successive, l’adozione di tecniche di scaling robuste agli outlier (**RobustScaler**).

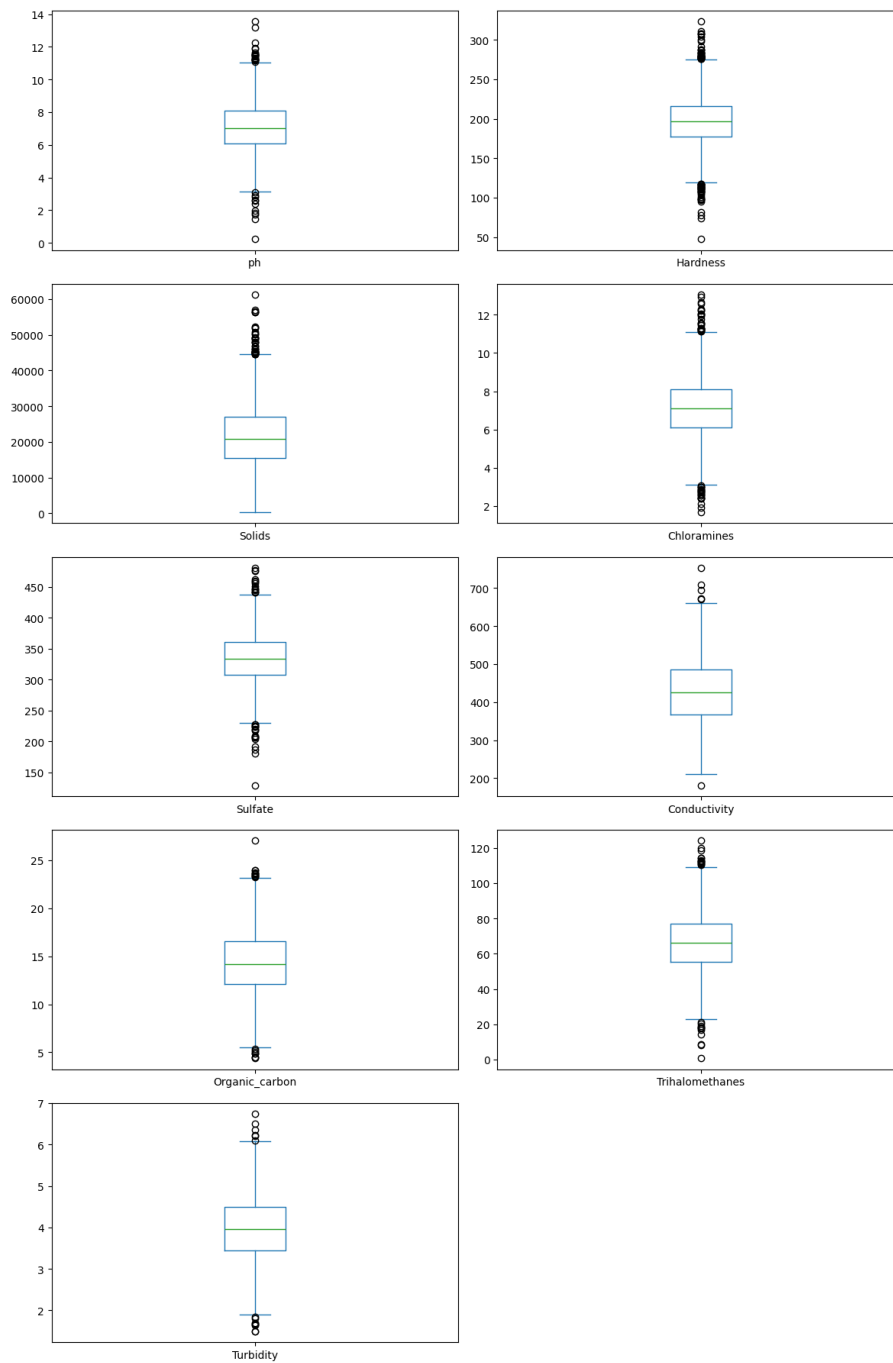


Figura 4: Boxplot delle feature.

2.4 Analisi delle correlazioni

La matrice di correlazione ha fornito il risultato più rilevante dell'intera analisi esplorativa, con due osservazioni principali.

1. In primo luogo, le **feature risultano debolmente correlate tra loro**: la correlazione massima osservata è di circa 0,22 in valore assoluto (tra le feature Solid e Sulfate). L'assenza di ridondanza implica che nessuna variabile è stata rimossa per collinearità.
2. In secondo luogo, la **correlazione tra ciascuna feature e la variabile target è praticamente nulla**. Nessuna variabile, considerata singolarmente, presenta dunque un'associazione lineare apprezzabile con la potabilità.

Questo risultato ha due implicazioni dirette sulle scelte di modellazione. Da un lato sconsiglia una *feature selection* basata sulla sola correlazione, che scarterebbe variabili potenzialmente informative solo se combinate tra loro. Dall'altro suggerisce che l'eventuale segnale predittivo non risieda in relazioni lineari, bensì in **interazioni non lineari** tra le feature. Tale ipotesi trova conferma nella fase di screening dei modelli (descritta nel paragrafo 4.1): come si osserverà, i risultati peggiori si registrano proprio con i modelli lineari, mentre le prestazioni migliori si ottengono con approcci capaci di cogliere relazioni non lineari e interazioni tra variabili, come SVM con kernel non lineare e metodi ensemble basati su alberi (Random Forest, Extra Trees, Gradient Boosting).

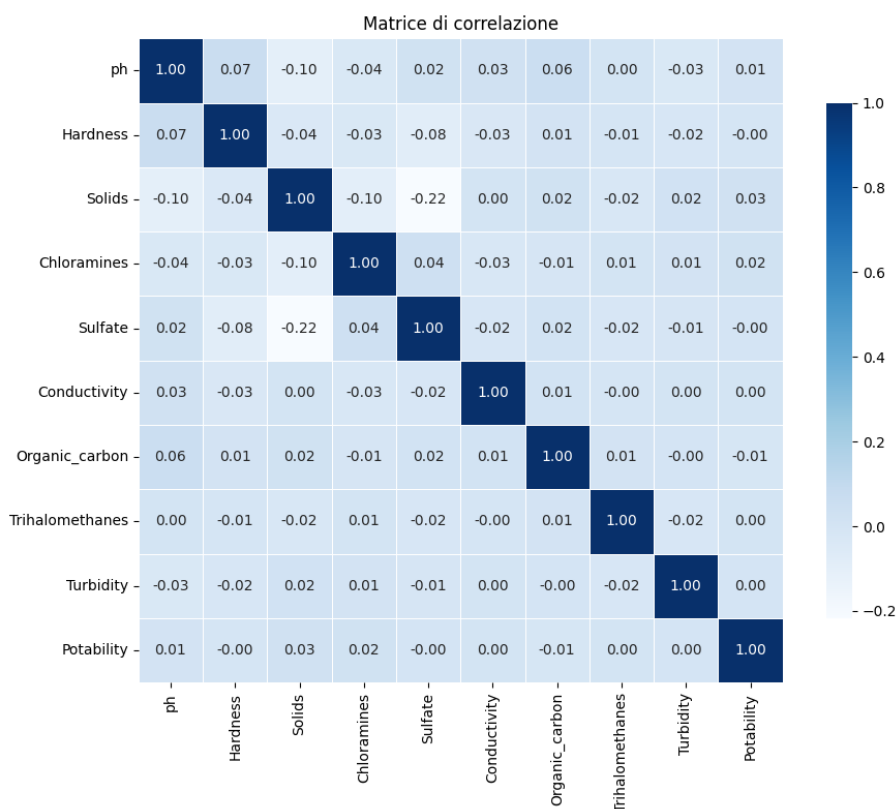


Figura 5: Matrice di correlazione.

2.5 Valori mancanti

L'analisi dei valori mancanti ha individuato tre sole feature interessate dal fenomeno: **Sulfate**, **ph** e **Trihalomethanes**. La distribuzione dei dati mancanti è stata valutata anche separatamente per classe, allo scopo di verificare l'eventuale introduzione di un bias sistematico. I valori mancanti risultano distribuiti in modo sostanzialmente uniforme tra le due classi — con una lieve prevalenza nella classe “non potabile” — senza tuttavia configurare uno sbilanciamento tale da orientare l'imputazione verso una delle due classi.

ph Sulfate Trihalomethanes			
Potability			
0	15.5	24.6	5.7
1	13.9	23.2	4.1

Figura 6: Distribuzione dei valori mancanti per classe.

Si è inoltre osservato che le mediane di tali feature, calcolate separatamente per classe, sono quasi identiche. Questo elemento, unito alla considerazione che la classe non è un'informazione disponibile per le nuove istanze da classificare, ha portato a scartare l'imputazione condizionata alla classe, che costituirebbe una forma di leakage. Per questi motivi si è scelto di adottare una strategia di imputazione globale, la cui implementazione è descritta nel capitolo successivo.

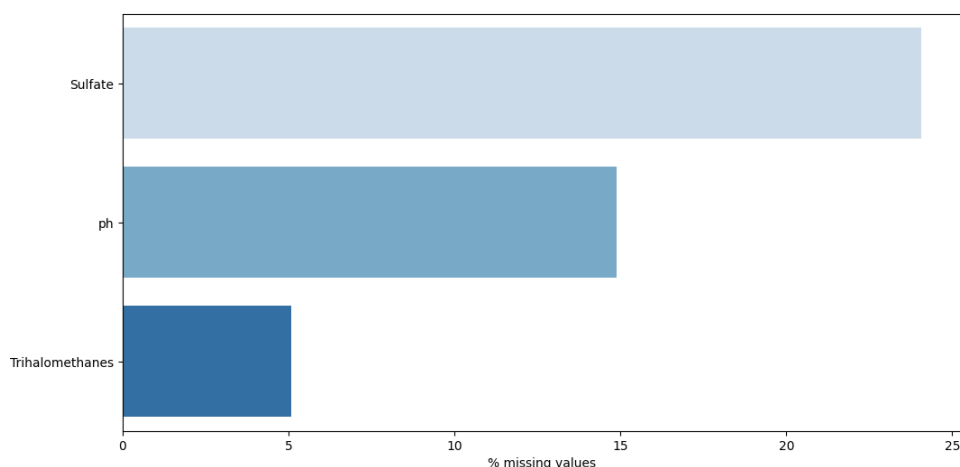


Figura 7: Percentuale dei valori mancanti.

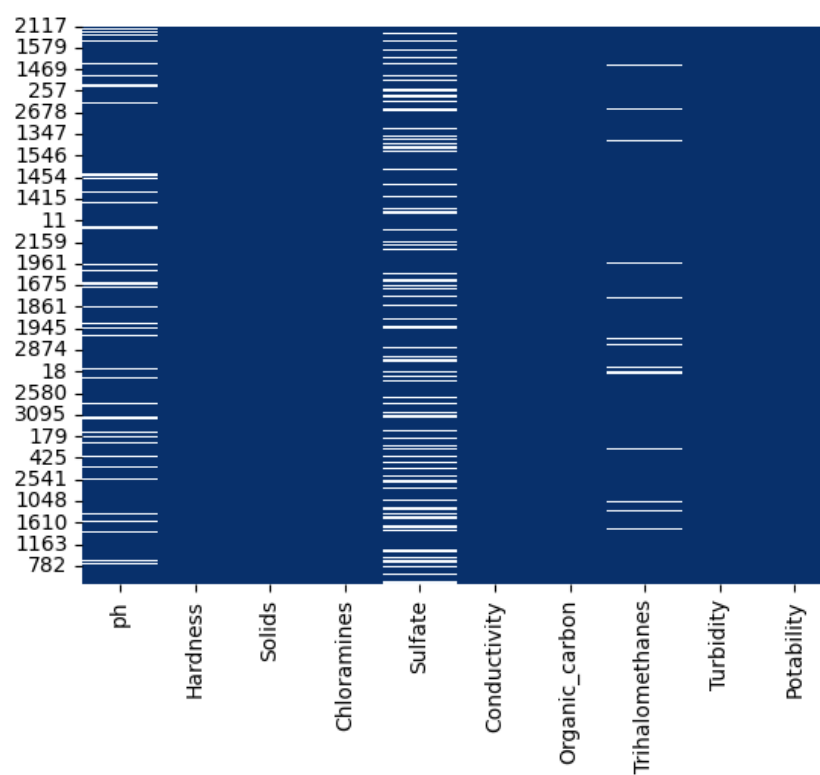


Figura 8: Analisi dei valori mancanti.

3 Preprocessing e preparazione dei dati

Una volta completata l'analisi esplorativa, i dati sono stati sottoposti alle trasformazioni necessarie a renderli idonei al training dei modelli.

3.1 Pipeline

Imputazione e scaling stimano dei parametri dai dati: la mediana di una colonna nel primo caso, mediana e intervallo interquartile nel secondo. Calcolare tali parametri una sola volta sull'intero training set, prima della *k-fold cross-validation*, costituirebbe una forma sottile di leakage, poiché le statistiche userebbero anche le osservazioni che, fold dopo fold, fungono da insieme di validazione.

Per evitare questo problema, ogni fase di preprocessing è stata incapsulata, insieme al modello, all'interno di una **Pipeline**. In questo modo, durante la cross-validation, le trasformazioni vengono ricalibrate (**fit**) esclusivamente sulla porzione di training di ciascun fold e poi applicate (**transform**) alla porzione di validazione, riproducendo correttamente la condizione in cui ci si troverebbe di fronte a dati realmente nuovi. La Pipeline diventa così non solo uno strumento di organizzazione del codice, ma la garanzia metodologica che impedisce la contaminazione tra training e validazione lungo tutto il processo.

3.2 Imputazione dei missing value

Come emerso dall'analisi esplorativa, i valori mancanti interessano tre feature: **ph**, **Sulfate** e **Trihalomethanes**. Trattandosi di un problema di classificazione in cui le istanze incomplete rappresentano una frazione non trascurabile del dataset, la loro eliminazione avrebbe comportato una perdita eccessiva di informazione; si è quindi optato per l'imputazione.

La sostituzione è stata effettuata con la **mediana** di ciascuna feature, calcolata sul solo training set all'interno della Pipeline. La scelta della mediana, in luogo della media, è motivata dalla presenza di outlier: essendo una misura robusta, la mediana non viene distorta dalle code delle distribuzioni e fornisce quindi una stima più rappresentativa del valore centrale. Coerentemente con quanto discusso nel capitolo precedente, l'imputazione è di tipo **globale** e non condizionata alla classe, sia perché le mediane per classe risultavano sostanzialmente coincidenti, sia perché l'etichetta non è disponibile al momento della classificazione di nuove istanze.

3.3 Scaling delle feature

Le nove variabili presentano scale diverse tra loro: i **Solids** assumono valori dell'ordine delle decine di migliaia, mentre il **ph** e la **Turbidity** si collocano entro poche unità. Diversi algoritmi come SVM e reti neurali sono sensibili a tali differenze di scala, che rischierebbero di far dominare arbitrariamente le feature con valori numericamente più grandi.

Per uniformare le scale è stato adottato il **RobustScaler**. A differenza della standardizzazione classica, che utilizza media e deviazione standard, il **RobustScaler** centra i dati sulla mediana e li riscalda in base all'intervallo interquartile. Lo scaling è stato applicato unicamente ai modelli che ne traggono beneficio, mentre è stato omesso per gli altri metodi (Random Forest, Extra Trees, Gradient Boosting, XGBoost).

3.4 Gestione dello sbilanciamento delle classi

Il dataset presenta un lieve sbilanciamento tra le classi, con circa il 60% delle istanze non potabili e il 40% potabili. Pur non trattandosi di uno sbilanciamento severo, esso è sufficiente a indurre i modelli a privilegiare la classe maggioritaria, riducendo la capacità di riconoscere correttamente i campioni potabili.

Per contrastare lo sbilanciamento sono state adottate strategie diverse a seconda del modello. Per Random Forest e SVM si è impostato `class_weight='balanced'`, che assegna a ciascuna classe un peso inversamente proporzionale alla sua frequenza, facendo pesare maggiormente gli errori sulla classe minoritaria; per XGBoost si è usato l'analogo `scale_pos_weight`. Questi approcci agiscono a livello di funzione di costo. Per l'MLP, che non supporta i pesi di classe, si è invece operato a livello di dati tramite oversampling SMOTE della classe minoritaria; la rete in Keras, infine, è stata riequilibrata con pesi di classe calcolati sul training. In tutti i casi il riequilibrio è stato confinato alla sola fase di addestramento, all'interno di pipeline o di cicli di cross-validation che lo applicano fold per fold, senza intaccare validazione e test.

4 Modelli

4.1 Scelta dei modelli da ottimizzare

Prima di procedere all'ottimizzazione degli iperparametri, è stata condotta una fase di *screening* su un'ampia gamma di modelli, addestrati con configurazione di default e valutati tramite *k-fold cross-validation* sul training set, con la metrica F1 macro sulle fold di validazione. L'obiettivo di questa fase non era ottenere le prestazioni finali, bensì individuare quali famiglie di algoritmi rispondessero meglio alla natura del problema, così da concentrare il successivo sforzo computazionale solo sui candidati più promettenti.

I risultati hanno mostrato un quadro coerente con quanto emerso dall'analisi esplorativa. I modelli **lineari** (SVM lineare, SVM polinomiale) e AdaBoost si sono collocati nettamente in coda alla classifica, confermando l'inadeguatezza dell'utilizzo di confini decisionali lineari per questo dataset. Le prestazioni migliori sono invece state ottenute da modelli capaci di catturare relazioni non lineari e interazioni tra le feature: Multi Layer Perceptron, Random Forest, XGBoost e l'SVM con kernel RBF.

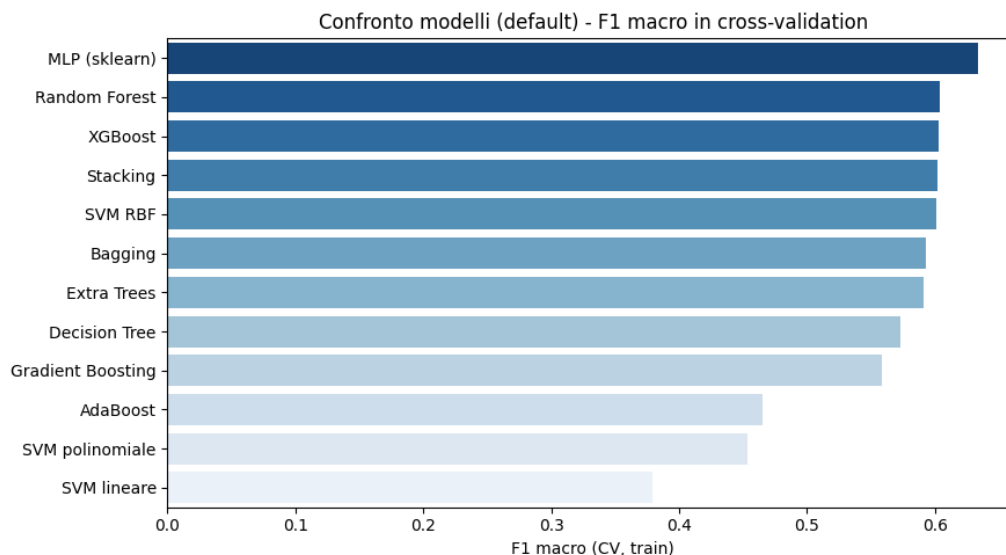


Figura 9: Screening dei modelli (F1 macro in cross-validation).

Sulla base di questa graduatoria sono stati selezionati per l'ottimizzazione cinque modelli, scelti in modo da rappresentare i principali paradigmi di apprendimento risultati efficaci: **Random Forest**, **XGBoost**, **SVM RBF**, **MLP** e una **rete neurale** implementata in Keras.

4.2 Scelta iperparametri con Grid Search

Per quattro dei cinque modelli selezionati l'ottimizzazione degli iperparametri è stata effettuata mediante **Grid Search con cross-validation**. La tecnica esplora in modo esaustivo una griglia di valori definita manualmente per ciascun iperparametro: per ogni combinazione possibile, il modello viene addestrato e valutato in *k-fold cross-validation*, e viene infine selezionata la configurazione che massimizza la metrica F1 macro. Metodo implementato nel seguente modo:

```

1 def run_grid_search(pipe, param_grid, scoring='f1_macro', n_splits=5):
2     cv = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=
        RANDOM_STATE)
3
4     grid = GridSearchCV(
5         pipe,
6         param_grid,
7         scoring=scoring,
8         cv=cv,
9         n_jobs=-1)
10
11     grid.fit(X_train, y_train)
12
13     print(f"Combinazioni provate: {len(grid.cv_results_['params'])}")
14     print(f"Miglior {scoring} (CV): {grid.best_score_:.3f}")
15     print("Migliori parametri:")
16     for k, v in grid.best_params_.items():
17         print(f"    {k}: {v}")
18     return grid

```

Listing 1: Funzione di Grid Search con cross-validation.

Coerentemente con quanto descritto nel capitolo sul preprocessing, l'intera ricerca è stata eseguita all'interno della Pipeline: imputazione e scaling vengono ricalibrati a ogni *fold* sulla sola porzione di training, garantendo che la selezione degli iperparametri non sia in alcun modo contaminata dai dati di validazione.

Ogni modello è stato addestrato sia sul dataset non bilanciato sia sul dataset bilanciato e nei paragrafi successivi andiamo ad analizzare i risultati ottenuti.

4.3 Random Forest

Per il modello Random Forest con dataset non bilanciato sono state provate 162 combinazioni di iperparametri, la configurazione ottimale prevede:

- 400 alberi;
- profondità non vincolata (`max_depth: None`);
- criterio di splitting `gini`;
- un sottoinsieme del 70% delle feature considerato a ogni divisione (`max_features = 0.7`);
- un minimo di 3 campioni per foglia (`min_samples_leaf = 3`).

Tabella 1: Risultati Random Forest Non Bilanciato
(CV F1: 0.619 tuned / 0.604 non tuned — Tempo: 1 ora)

Metrica	Precision	Recall	F1-score	Support
Non Potabile	0.667	0.884	0.760	499
Potabile	0.633	0.312	0.418	320
Accuracy			0.661	819
Macro avg	0.650	0.598	0.589	819
Weighted avg	0.654	0.661	0.627	819

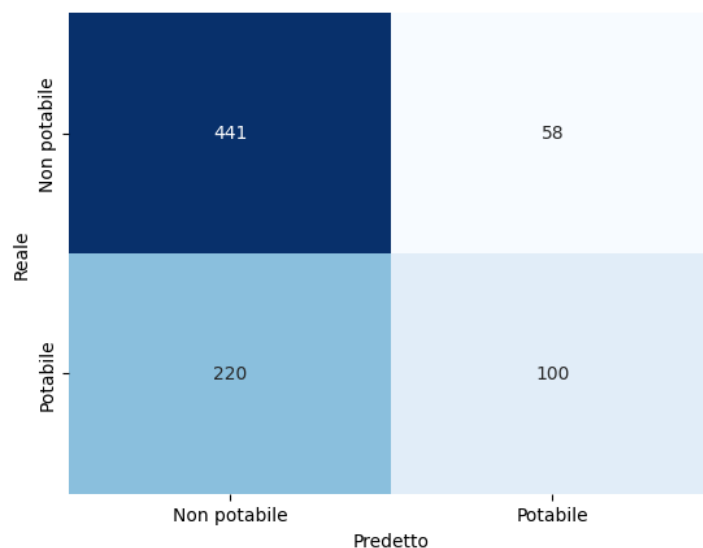


Figura 10: Matrice di confusione — Random Forest non bilanciato.

Per il modello Random Forest con dataset bilanciato sono state provate 162 combinazioni di iperparametri, la configurazione ottimale prevede:

- 200 alberi;
- profondità non vincolata (`max_depth: None`);
- criterio di *splitting* gini;
- un sottoinsieme del 70% delle feature considerato a ogni divisione (`max_features = 0.7`);
- un minimo di 8 campioni per foglia (`min_samples_leaf = 8`).

Tabella 2: Risultati Random Forest Bilanciato
(CV F1: 0.632 tuned / 0.604 non tuned — Tempo: 1 ora)

Metrica	Precision	Recall	F1-score	Support
Non Potabile	0.694	0.814	0.749	499
Potabile	0.603	0.441	0.509	320
Accuracy			0.668	819
Macro avg	0.648	0.627	0.629	819
Weighted avg	0.658	0.668	0.655	819

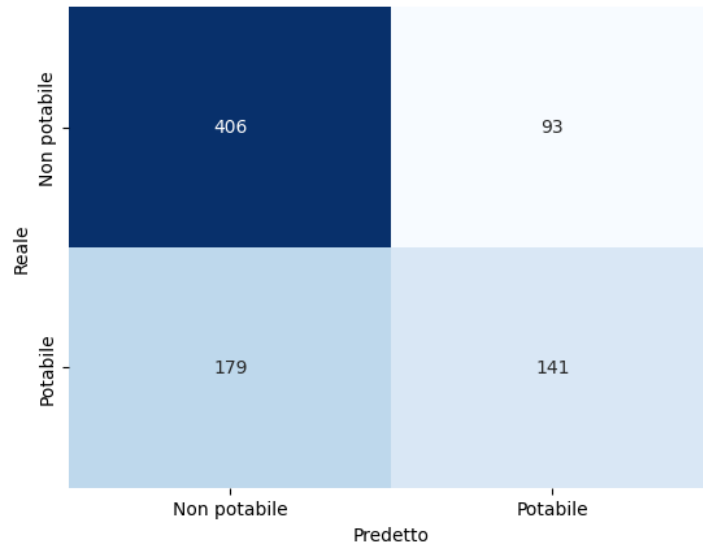


Figura 11: Matrice di confusione — Random Forest bilanciato.

4.4 XGBoost

Per il modello XGBoost con dataset non bilanciato sono state provate 128 combinazioni di iperparametri, la configurazione ottimale prevede:

- 200 stimatori;
- *learning rate* basso (`learning_rate` = 0.03);
- compensato da alberi profondi (`max_depth` = 10);
- *subsampling* del 100% delle istanze a ogni iterazione (`subsample` = 1).

Tabella 3: Risultati XGBoost Non Bilanciato
(CV F1: 0.638 tuned / 0.603 non tuned — Tempo: 8 minuti)

Metrica	Precision	Recall	F1-score	Support
Non Potabile	0.677	0.848	0.753	499
Potabile	0.608	0.369	0.459	320
Accuracy			0.661	819
Macro avg	0.643	0.608	0.606	819
Weighted avg	0.650	0.661	0.638	819

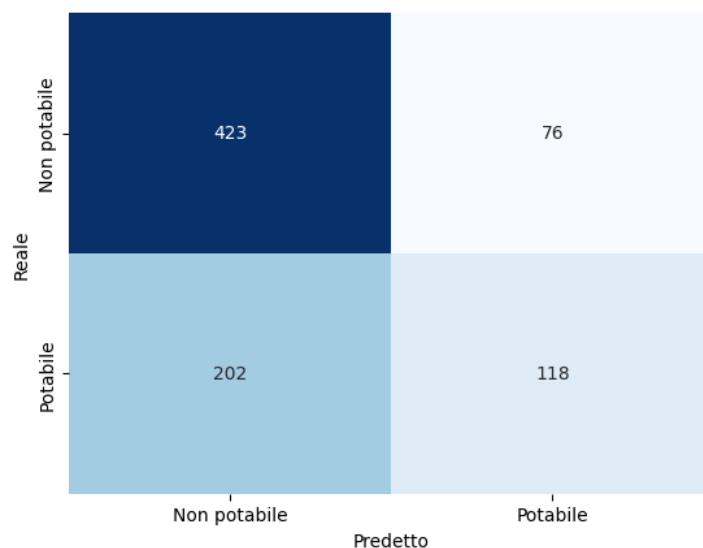


Figura 12: Matrice di confusione — XGBoost non bilanciato.

Per il modello XGBoost con dataset bilanciato sono state provate 128 combinazioni di iperparametri, la configurazione ottimale prevede:

- 100 stimatori;
- *learning rate* basso (`learning_rate` = 0.02);
- compensato da alberi profondi (`max_depth` = 10);
- *subsampling* dell'80% delle istanze a ogni iterazione (`subsample` = 0.8).

Tabella 4: Risultati XGBoost Bilanciato
(CV F1: 0.638 tuned / 0.603 non tuned — Tempo: 8 minuti)

Metrica	Precision	Recall	F1-score	Support
Non Potabile	0.693	0.760	0.725	499
Potabile	0.559	0.475	0.514	320
Accuracy			0.648	819
Macro avg	0.626	0.617	0.619	819
Weighted avg	0.640	0.648	0.642	819

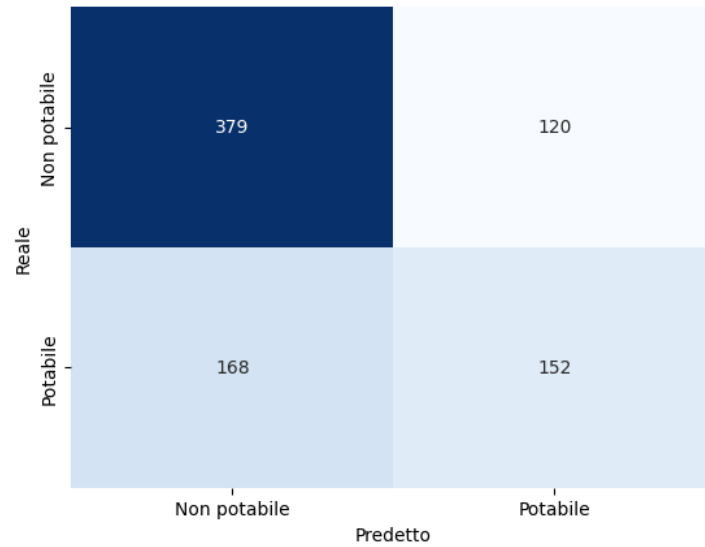


Figura 13: Matrice di confusione — XGBoost bilanciato.

4.5 SVM RBF

Per il modello SVM con kernel RBF con dataset non bilanciato sono state provate 20 combinazioni di iperparametri, la configurazione ottimale prevede:

- parametro di regolarizzazione $C = 10$;
- coefficiente del kernel $\gamma = 0.1$.

Tabella 5: Risultati RBF Non Bilanciato
(CV F1: 0.642 tuned / 0.601 non tuned — Tempo: 42 secondi)

Metrica	Precision	Recall	F1-score	Support
Non Potabile	0.677	0.818	0.740	499
Potabile	0.579	0.391	0.466	320
Accuracy			0.651	819
Macro avg	0.628	0.604	0.603	819
Weighted avg	0.638	0.651	0.633	819

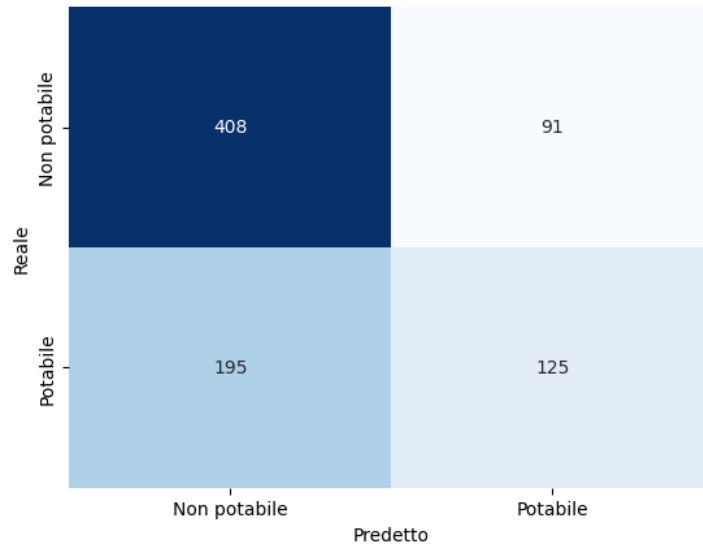


Figura 14: Matrice di confusione — SVM RBF non bilanciato.

Per il modello SVM con kernel RBF con dataset bilanciato sono state provate 20 combinazioni di iperparametri, la configurazione ottimale prevede:

- parametro di regolarizzazione $C = 10$;
- coefficiente del kernel $\gamma = 0.1$.

Tabella 6: Risultati SVM RBF Bilanciato
(CV F1: 0.647 tuned / 0.601 non tuned — Tempo: 37 secondi)

Metrica	Precision	Recall	F1-score	Support
Non Potabile	0.685	0.713	0.699	499
Potabile	0.522	0.487	0.504	320
Accuracy			0.625	819
Macro avg	0.603	0.600	0.601	819
Weighted avg	0.621	0.625	0.623	819

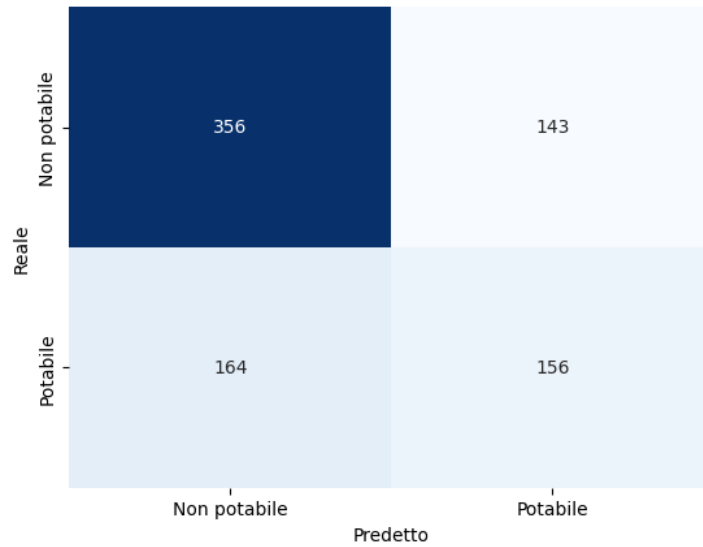


Figura 15: Matrice di confusione — SVM RBF bilanciato.

4.6 MLP

Per il modello Multi Layer Perceptron con dataset non bilanciato sono state provate 96 combinazioni di iperparametri, la configurazione ottimale prevede:

- parametro di regolarizzazione `alpha = 0.1`;
- due strati nascosti da 64 e 32 neuroni (`hidden_layer_sizes = (64, 32)`);
- funzione di attivazione `relu`;
- stop anticipato (`early_stopping: True`).

Tabella 7: Risultati MLP Non Bilanciato
(CV F1: 0.632 tuned / 0.596 non tuned — Tempo: 3 minuti)

Metrica	Precision	Recall	F1-score	Support
Non Potabile	0.659	0.932	0.772	499
Potabile	0.699	0.247	0.365	320
Accuracy			0.664	819
Macro avg	0.679	0.589	0.568	819
Weighted avg	0.674	0.664	0.613	819

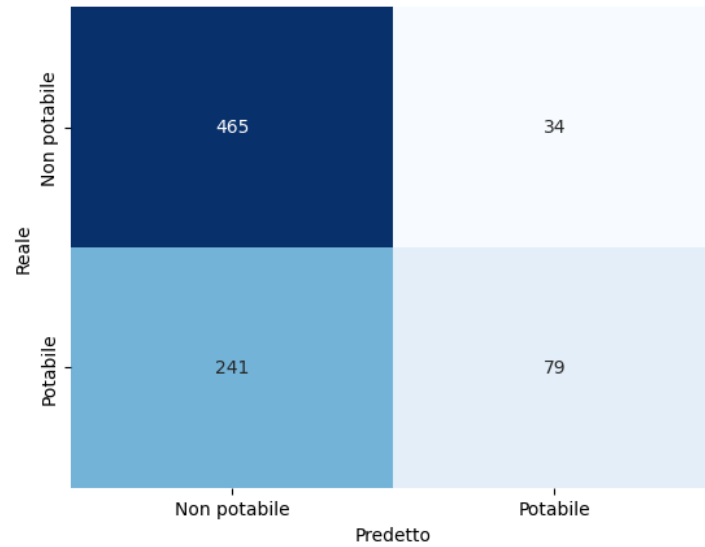


Figura 16: Matrice di confusione — MLP non bilanciato.

Per il modello Multi Layer Perceptron con dataset bilanciato sono state provate 96 combinazioni di iperparametri, la configurazione ottimale prevede:

- parametro di regolarizzazione `alpha = 0.001`;
- due strati nascosti da 128 e 64 neuroni (`hidden_layer_sizes = (128, 64)`);
- funzione di attivazione `relu`;
- stop anticipato (`early_stopping: True`).

Tabella 8: Risultati MLP Bilanciato
(CV F1: 0.641 tuned / 0.634 non tuned — Tempo: 5 minuti)

Metrica	Precision	Recall	F1-score	Support
Non Potabile	0.683	0.661	0.672	499
Potabile	0.497	0.522	0.509	320
Accuracy			0.607	819
Macro avg	0.590	0.592	0.591	819
Weighted avg	0.610	0.607	0.608	819

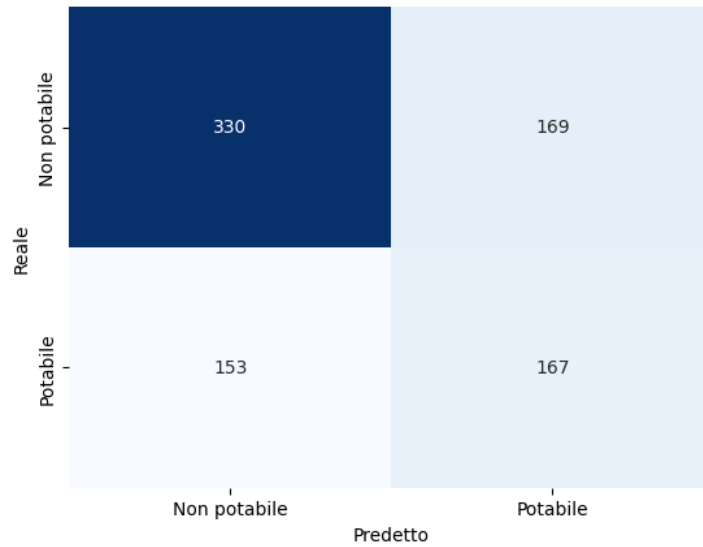


Figura 17: Matrice di confusione — MLP bilanciato.

4.7 Rete neurale con Optuna

Per il modello Rete Neurale con dataset non bilanciato sono state provate 30 trial, la configurazione ottimale prevede:

- 2 strati nascosti da 128 neuroni;
- un *dropout* di circa 0,2;
- *learning rate* di circa 0,0021;
- 100 epoche;
- *batch* di dimensione 64.

Tabella 9: Risultati Neural Network Non Bilanciata
(CV F1: 0.641 tuned — Tempo: 41 minuti)

Metrica	Precision	Recall	F1-score	Support
Non Potabile	0.670	0.749	0.708	499
Potabile	0.521	0.425	0.468	320
Accuracy			0.623	819
Macro avg	0.596	0.587	0.588	819
Weighted avg	0.612	0.623	0.614	819

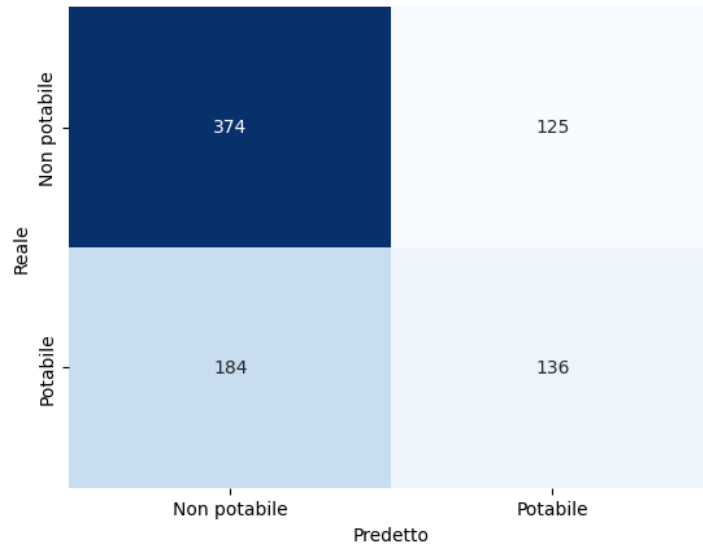


Figura 18: Matrice di confusione — Neural Network non bilanciata.

Per il modello Rete Neurale con dataset bilanciato sono state provate 30 trial, la configurazione ottimale prevede:

- 3 strati nascosti da 128 neuroni;
- un *dropout* di 0,3;
- *learning rate* di circa 0,0013;
- 100 epoche;
- *batch* di dimensione 64.

Tabella 10: Risultati Neural Network Bilanciata
(CV F1: 0.637 tuned — Tempo: 54 minuti)

Metrica	Precision	Recall	F1-score	Support
Non Potabile	0.688	0.697	0.693	499
Potabile	0.518	0.506	0.512	320
Accuracy			0.623	819
Macro avg	0.603	0.602	0.602	819
Weighted avg	0.621	0.623	0.622	819

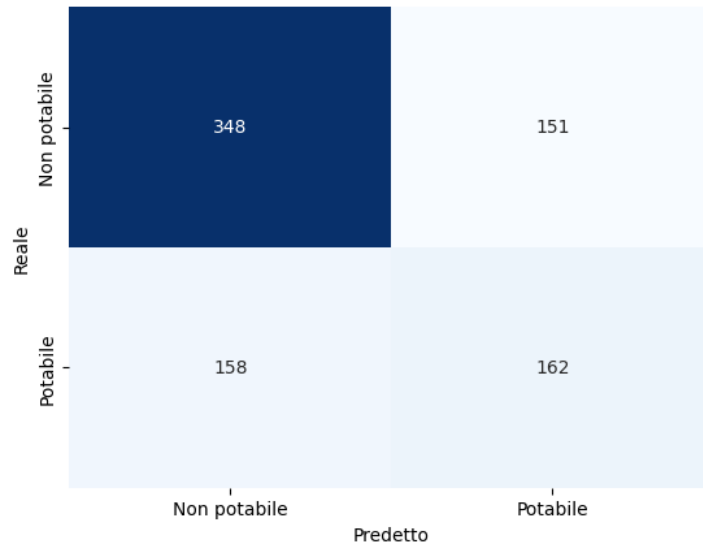


Figura 19: Matrice di confusione — Neural Network bilanciata.

4.8 Confronto modelli con e senza bilanciamento

Il confronto tra le due configurazioni mette in luce l’effetto del bilanciamento delle classi sul comportamento dei modelli. Senza bilanciamento tutti i modelli mostrano lo stesso schema: una precision elevata sulla classe “Potabile” a fronte di una recall molto bassa (tra il 25% e il 42%). I modelli classificano cioè la maggior parte dei campioni come “Non potabile” (classe maggioritaria) individuando solo una piccola frazione delle acque effettivamente potabili. L’accuracy resta apparentemente buona (intorno al 66%) proprio perché premia la predizione della classe più frequente, ma questo dato maschera la scarsa capacità di riconoscere la classe minoritaria.

L’introduzione del bilanciamento della classe ribalta questo equilibrio. La recall sulla classe “Potabile” aumenta sensibilmente in tutti i modelli (portandosi tra il 44% e il 52%), a fronte di un calo contenuto della precision. L’effetto netto è un miglioramento dell’F1 macro sul test per quasi tutti i modelli, mentre l’accuracy resta stabile o diminuisce leggermente. Questo apparente paradosso è in realtà coerente: il bilanciamento “sacrifica” alcune predizioni facili sulla classe maggioritaria per recuperare quella minoritaria, spostando il modello verso un comportamento più equo e quindi verso un F1 macro più alto.

Tabella 11: Modelli senza bilanciamento.

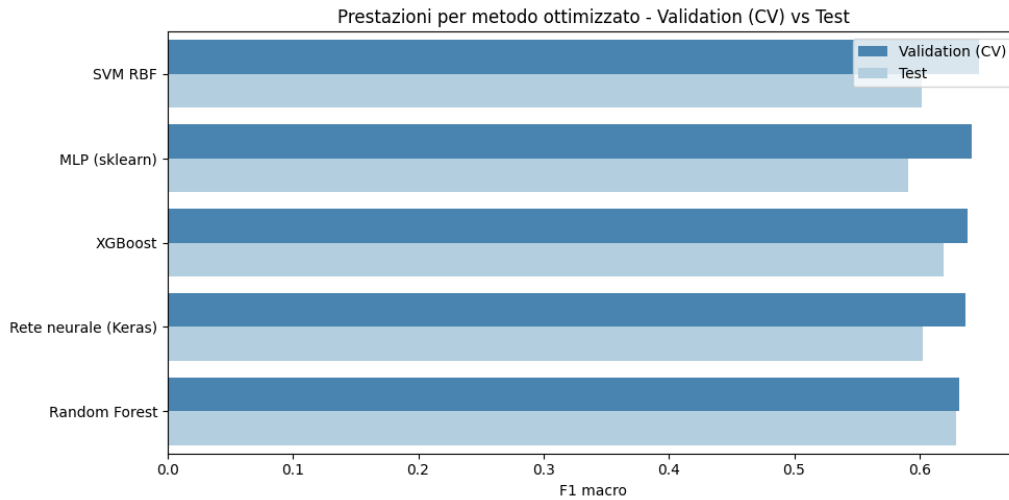
	RF	XGB	SVM RBF	MLP	NN
Validation F1	61.9%	63.8%	64.2%	63.2%	64.1%
Test F1	58.9%	60.6%	60.3%	56.8%	58.8%
Precision	63.3%	60.8%	57.9%	69.9%	52.1%
Recall	31.2%	36.9%	39.1%	24.7%	42.5%
Accuracy	66.1%	66.1%	65.1%	66.4%	62.3%

Tabella 12: Modelli con bilanciamento.

	RF	XGB	SVM RBF	MLP	NN
Validation F1	63.2%	63.8%	64.7%	64.1%	63.7%
Test F1	62.9%	61.9%	60.1%	59.1%	60.2%
Precision	60.3%	55.9%	52.2%	49.7%	51.8%
Recall	44.1%	47.5%	48.7%	52.2%	50.6%
Accuracy	66.8%	64.8%	62.5%	60.7%	62.3%

4.9 Modello migliore

Pur nella sostanziale equivalenza tra i modelli, è possibile individuare un candidato preferibile. La selezione, condotta correttamente sul punteggio in cross-validation, premia SVM RBF (0,647); tuttavia tra tutti i modelli ottimizzati, la **Random Forest bilanciata** offre il miglior compromesso complessivo: ottiene l’F1 macro più alto sul test (0,629) ed è al tempo stesso il modello con il minor divario tra cross-validation e test (0,632 contro 0,629), segno di una generalizzazione particolarmente stabile e di un overfitting contenuto. Resta però inteso che, dato il limite intrinseco del dataset, tali differenze rientrano in larga parte nel margine di rumore e non vanno sovrainterpretate.

**Figura 20:** Confronto finale tra i modelli.

5 Conclusioni

L'obiettivo di questo lavoro era la classificazione della potabilità dell'acqua a partire da parametri chimico-fisici. L'analisi esplorativa ha però evidenziato un limite intrinseco del dataset, verosimilmente sintetico: la correlazione tra feature e target è risultata pressoché nulla, con le classi ampiamente sovrapposte. I dati non contengono quindi informazione sufficiente a costruire una regola di classificazione generalizzabile.

Coerentemente, tutti i modelli (Random Forest, XGBoost, SVM, MLP e rete neurale) hanno raggiunto prestazioni simili, attorno a un F1 macro di circa 0.63 sul test, nonostante un'estesa ottimizzazione degli iperparametri il cui guadagno rispetto ai default è stato marginale.

Centrale è stato il rigore metodologico per non sovrastimare le prestazioni: split prima del preprocessing, con imputazione e scaling appresi solo sul training dentro pipeline e ricalcolati in ogni fold, evitando ogni data leakage. Questo si è rivelato decisivo, poiché diversi notebook di riferimento su Colab raggiungono accuratezze elevate (fino all'81%) che derivano in realtà da imputazioni condizionate alla classe eseguite prima dello split, il che gonfia i risultati.

In conclusione, questo lavoro mostra che non sempre è possibile ottenere buone prestazioni predittive, anche con tecniche avanzate e una pipeline accuratamente ottimizzata, quando il dataset presenta un limite intrinseco. Il valore del progetto non sta nel raggiungere un'accuratezza elevata, ma nell'aver diagnosticato in modo rigoroso e documentato l'assenza di segnale nei dati, distinguendo le prestazioni reali da quelle apparenti prodotte da scorciatoie metodologiche.